

Jordan University of Science & Technology
Department of Network Engineering and Security
NES416- Network Programming
Programming Assignment 2 (CLO2, CLO3)

Due Date: see Elearning

Goal: Understand how to build a concurrent server and deal with signals

Description:

Use TCP socket programming in C to implement a pair of client and server programs. When the client runs (you need to supply IP address and port number of the server through the command line), it simply asks the user to enter a string (null-terminated), and then displays a menu for the user to select an operation on that string. The string and the type of operation are sent to the child server which performs the required operation and returns the result back to the client.

The server will listen to a port entered through the command line. Once it receives a request, it forks a child and to perform the request, and prints the port and IP address of the client, while the parent is waiting for more requests. Then it (the parent) waits for the child to finish processing client's request and prints the process ID of the terminated child.

The connection between your client and (child) server should stay open so that another request can be sent (re-display the menu) or until the user sends (generates) the SIGQUIT signal, at which time the client's handler will print a notifying message on the clients terminal (for example, "Done") before it (the client) exits. Note that in all cases, the parent server is kept running.

The menu displayed to the user (client side) is composed of the following choices:

1. Change the string to capital letters
2. Count number of characters in the string
3. Count the frequency of some character (the user has to enter a character)
4. Enter another string

The server once receives the string, performs the required operation and returns the result back to the client waiting for another request. The string should be at most 80 characters

The request messages from the client and the server should follow this format

Request type (one bytes)	Args (one byte)	String (variable length)
-----------------------------	--------------------	-----------------------------

The request is a one-byte representing the operation that is required. Its value depends on the choices of the menu. For example, if the client wants to count the characters in the string, the request value will be 2, and so on. The Args is a one-byte representing the arguments of some of the requests. For example, if the clients asks for the frequency of occurrence of the letter b in the string, the request value will be 3 and the Args is the desired character. If the request requires no argument this field is unused. The string field contains the actual sent string.

The reply messages from the server to the client should follow this format

Type (one bytes)	result (variable length)
---------------------	-----------------------------

The reply type is a one-byte value that represents the status of the reply. Success (0) or error (1). If success, the result field contains the answer of the requested operation. Otherwise, it contains an error message indicating the error that occurred. For example, the client might ask for a character that doesn't exist in the string. Remember that **the (child) server should prints the IP and port number from which it receives the request**

Output Sample:

For example, running the code should produce something similar to this output:

Client_side> please enter a string:

hello students (input from a user)

Client_side > please select an operation from the following menu (menu is displayed for the user...)

Client_side> sending "hello students" to the server with requested operation 1

Client_side > received "HELLO STUDENTS" from the server (if an error occurred, then it is displayed)

Client_side > please select an operation from the following menu (menu is displayed for the user...)

And the same happens again until the user quits by generating the SIGQUIT. At the time it prints goodbye

Server_side (parent) > waiting for client messages.

Sever_side (parent) > ***created child process with PID =<...> to handle client***

Sever_side (child) > reveiced "hello students" from the client (IP address, port #) with request type=1

Server_side (child)> Sending " HELLO STUDENTS " to the client

Server_side (child) > waiting for client messages

Server_side (child) > received exit from client ...Goodbye (assuming the client exited)

Parent_server> child whose ID = <....> Has exited

Submission:

- You files should follow the following naming convention: yourID_HW#
- Submit a zipped file containing only the source code (following the naming convention) and a screen shot of compiling and running your code (both client and server)

Hints:

- ☐ DO NOT use the header file "unp.h" from the book
- ☐ This assignment is implement as a group of two students

- Only one member submit a solution
 - A penalty of -50 pts will be given if you work alone
- ❑ Each student/group should use a port number for the server that follows the following format:
 - Port No = 55xxx, where xxx is the first three digits of your student ID
 - Ex: if my student ID is 175**123**, then my port number is 55**123**.
- ❑ Your program for client needs to take two arguments that specify the IP address of the server and the port that it is trying to connect to. Your program for server needs to take an argument that specifies the port that it is listening to (Not hardcoded in the program)
- ❑ Don't use the bind() function on the client
- ❑ Ask questions as early as possible.
- ❑ Submit your source code for both the client and the server, and some running sample of your code as one zipped file whose name is your student ID number
- ❑ Your programs should be compiled and run without any single error or warning. TA will run your code assuming **Ubuntu-16.04.3 or 20.4**
- ❑ Comment and error-check your code. **Part of the comment is to include member names and their ID's**
- ❑ **Make sure zombies and interrupted system calls are handled correctly**
- ❑ **Make sure you have valid input before sending the data to the server**